

REPUBLIC OF CAMEROON
Peace – work – Fatherland
REPUBLIQUE DU CAMEROUN
Paix – Travail - Patrie
THE UNIVERSITY OF BAMENDA
P.O. Box 39, Bambili, Mezam Division, NW Region, Cameroon
HIGHER INSTITUTE OF TRANSPORT AND LOGISTICS (HITL)

DEPARTMENT OF DEPARTMENT OF COMPUTER ENGINEERING

**DESIGN AND IMPLEMENTATION OF LOCO: A
PRIVACY-PRESERVING LOCAL AI ASSISTANT WITH
INTEGRATED SPEECH AND LANGUAGE PROCESSING**

Assignment submitted in partial fulfilment of the requirements for the award of
MTech - Master of Technology

TAMANKAG ALA CHO PHILDING
Registration Number: UBa25PP167

SUPERVISOR(S):
Dr. [Lecturer Name]
APRIL 2026

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	1
1.1 Background and Context	1
1.2 Problem Statement	2
1.3 Research Questions	2
1.4 Objectives	3
1.5 Scope and Limitations	3
CHAPTER 2: LITERATURE REVIEW	4
2.1 Local AI Inference: Enabling Technologies	4
2.2 Large Language Models - Gemma 2b	4
2.3 Speech-to-Text - faster-whisper	5
2.4 Text-to-Speech - Piper ONNX	6
2.5 Ollama: Local LLM Serving Infrastructure	6
2.6 Privacy and Data Sovereignty in AI Applications	7
2.7 Multi-Modal AI Application Architecture	7
CHAPTER 3: SYSTEM DESIGN AND MATERIALS	8
3.1 System Architecture	8
3.2 Hardware and Software Requirements	8
3.3 Installation of Ollama	9
3.3.1 Linux Installation	9
3.3.2 Windows Installation	9
3.3.3 Pulling the Gemma 2b Model	10

3.4 Installation of the Python Voice Engine	10
3.4.2 Installing faster-whisper (STT)	11
3.4.3 Installing Piper TTS	11
3.4.4 Installing Flask and Additional Dependencies	11
3.5 Voice Engine Configuration	11
3.6 AI Model Specifications	12
3.7 Frontend Installation	12
3.8 System Launch	13
CHAPTER 4: SYSTEM IMPLEMENTATION	14
4.1 Frontend Implementation (page.tsx)	14
4.1.1 Streaming Chat Interface	14
4.1.2 Pipelined TTS Queue	15
4.1.3 Voice Recording and VAD.	15
4.2 API Gateway Implementation	15
Table 4.1: API Gateway Endpoint Specification	16
4.3 Voice Engine Implementation (main.py)	16
4.3.1 STT Endpoint (/stt)	16
4.3.2 TTS Endpoint (/tts)	17
4.3.3 STT Configuration Options	17
CHAPTER 5: RESULTS AND DISCUSSION	18
5.1 Performance Evaluation	18
5.2 Voice Quality Assessment	18

5.3 Hardware Resource Utilisation	19
5.4 Privacy Evaluation	19
5.5 Identified Limitations and Recommendations	20
5.5.1 Fixed Model Configuration	20
5.5.2 English-Only STT Configuration	20
5.5.3 Single Voice Profile	20
5.5.4 Process Supervision	20
CHAPTER 6: CONCLUSION, RECOMMENDATIONS AND PERSPECTIVES	22
6.1 Conclusion	22
6.2 Recommendations	23
6.3 Perspectives	24
REFERENCES	24
APPENDICES	27
Annex 1: Complete Installation Checklist	27
REFERENCES	28
APPENDICES	30

CHAPTER 1: INTRODUCTION

1.1 Background and Context

The rapid advancement of deep learning and neural network research over the past decade has produced increasingly capable AI systems for natural language processing, speech recognition, and voice synthesis. Historically, access to these capabilities required either expensive proprietary hardware or subscription-based cloud API services from providers including OpenAI, Google, Amazon, and Microsoft. While cloud-hosted AI services provide convenience and ease of integration, they introduce significant concerns regarding user privacy, data sovereignty, internet dependency, and ongoing subscription costs that scale with usage volume (Brown et al., 2020).

The emergence of open-source, locally executable AI tools marks a significant shift in the accessibility of AI technology. Tools such as Ollama, which provides a unified interface for running quantised LLMs on commodity CPU hardware, have lowered the barrier to local AI inference substantially. Similarly, faster-whisper provides CPU-optimised speech recognition based on OpenAI's Whisper architecture, and Piper TTS delivers neural-quality voice synthesis via the efficient ONNX Runtime - all without requiring cloud connectivity or API keys (Gerganov, 2023; Radford et al., 2023; Rhasspy Project, 2023).

Loco is a fully local AI assistant designed to integrate these technologies into a cohesive, production-quality application. Built on a Next.js 15 frontend, Next.js API gateway, and Python Flask voice engine, Loco delivers real-time streaming chat, voice input via Speech-to-Text, and voice output via Text-to-Speech - all executed on the user's own hardware. The name Loco reflects its core design philosophy: local computation, local data, local control.

This document presents a comprehensive account of the design rationale, system architecture, installation and configuration of all component services and AI models, implementation of key system modules, data flow mechanisms, performance characteristics, and recommendations for future development.

1.2 Problem Statement

Contemporary AI assistant applications predominantly depend on cloud infrastructure for core inference tasks. This creates four critical limitations for end users and organisations. First, persistent internet connectivity is required; any network disruption renders the assistant non-functional. Second, sensitive voice recordings and conversational content are transmitted to and processed by third-party service providers, creating privacy and compliance risks under frameworks such as GDPR and Cameroon's Law No. 2010/012 on Cybersecurity and Cybercriminality (Mireshghallah et al., 2020). Third, cloud API services impose per-request costs that scale proportionally with usage, making high-frequency AI assistant usage economically prohibitive for individuals and small organisations in developing economies.

Fourth, response quality and latency are subject to variability introduced by network conditions, server load, and geographic distance from cloud data centres.

While fully local AI execution has become technically feasible, practical multi-modal implementations integrating LLM, STT, and TTS within a single cohesive application remain underrepresented in available open-source projects. Loco directly addresses this gap by providing a documented, reproducible implementation of a fully local AI assistant with complete installation guidance for all component technologies.

1.3 Research Questions

- (i) How can Ollama, faster-whisper, and Piper TTS be integrated into a unified local AI assistant architecture that achieves sub-two-second end-to-end voice interaction latency?
- (ii) What installation and configuration procedures are required to deploy the Gemma 2b LLM, faster-whisper STT, and Piper ONNX TTS models on commodity hardware?
- (iii) How does the pipelined sentence-level TTS queuing mechanism improve perceived voice response naturalness during concurrent LLM token streaming?
- (iv) What are the performance characteristics of the Loco system in terms of voice quality, latency, and hardware resource utilisation?

1.4 Objectives

General Objective: To design and implement Loco, a fully local, privacy-preserving AI assistant integrating LLM, STT, and TTS capabilities on commodity hardware.

- (i) To design and document the three-layer distributed local architecture of the Loco system.
- (ii) To install, configure, and document all AI model components including Ollama, Gemma 2b, faster-whisper, and Piper ONNX.
- (iii) To implement the streaming chat interface, pipelined TTS queue, and voice recording subsystems.
- (iv) To evaluate system performance including voice quality, component latency, and hardware resource utilisation.
- (v) To identify architectural limitations and provide evidence-based recommendations for enhancement.

1.5 Scope and Limitations

This study encompasses the complete design, implementation, and documentation of the Loco system, including detailed installation procedures for all component services and AI models. The system is implemented for deployment on Linux and Windows environments with CPU-only inference, and tested on hardware representative of a typical developer workstation. The study does not include GPU-accelerated inference benchmarks, production cloud deployment, or multi-user concurrent access scenarios. Voice quality evaluation is based on subjective MOS scoring and objective RTP stream analysis within a controlled local network environment.

CHAPTER 2: LITERATURE REVIEW

2.1 Local AI Inference: Enabling Technologies

The practical feasibility of local AI inference has been enabled by three converging technological developments. First, model quantisation techniques reduce the numerical precision of neural network weights from 32-bit floating point to 8-bit or 4-bit integers, enabling models with billions of parameters to execute on consumer RAM without GPU acceleration. Frantar et al. (2022) demonstrated that 4-bit quantised LLMs can achieve inference quality comparable to full-precision counterparts at less than one quarter of the memory footprint. Second, the ONNX Runtime provides a hardware-agnostic, highly optimised inference engine supporting CPU, GPU, and dedicated AI accelerator targets through a single standardised model format. Third, unified model serving tools such as Ollama abstract the complexity of quantised model management, providing a simple CLI and REST API for downloading, managing, and serving locally executed LLMs (Gerganov, 2023).

2.2 Large Language Models - Gemma 2b

Large Language Models (LLMs) based on the transformer architecture introduced by Vaswani et al. (2017) achieve human-like text generation through autoregressive next-token prediction trained on large text corpora. The Gemma family of models, introduced by Google DeepMind (2024), represents a series of lightweight open models derived from the research underpinning the Gemini model series. Gemma 2b, with 2.51 billion parameters, is designed specifically for efficient deployment on resource-constrained devices including laptops and single-board computers. In GGUF Q4_K_M quantised format as served by Ollama, Gemma 2b occupies approximately 1.7 GB of storage and requires roughly 3.5 GB of RAM during inference, achieving first-token latency of 400-800 ms on a modern quad-core CPU (Google DeepMind, 2024).

Ollama provides a streaming REST API compatible with the OpenAI API schema, enabling LLM responses to be streamed token by token via Server-Sent Events (SSE). This streaming interface is exploited by the Loco frontend to render tokens in real time as they are generated, substantially reducing perceived response latency regardless of total response length (Holtzman et al., 2020).

2.3 Speech-to-Text - faster-whisper

Automatic Speech Recognition (ASR) has been transformed by the Whisper model family introduced by Radford et al. (2023), which established new benchmarks for multilingual transcription accuracy through large-scale weak supervision on 680,000 hours of audio. faster-whisper is a reimplementation of Whisper using the CTranslate2 inference engine, achieving up to 4x speedup over the original PyTorch implementation through int8 quantisation and optimised CPU memory access patterns (Bain et al., 2023). The tiny model variant (39M

parameters) provides the best trade-off between inference speed and transcription accuracy for conversational AI applications, achieving Word Error Rates (WER) of 8-12% on standard English benchmarks.

Voice Activity Detection (VAD) is integrated into Loco via the Silero VAD model, which pre-filters audio input to identify speech segments before passing them to the Whisper model. This reduces spurious transcription of background noise and silence, improving both accuracy and latency (Silero Team, 2021). The faster-whisper configuration used in Loco (beam_size=1, vad_filter=True, compute_type='int8') prioritises minimum latency through greedy decoding while maintaining acceptable accuracy for conversational input.

2.4 Text-to-Speech - Piper ONNX

Neural text-to-speech synthesis has achieved naturalness comparable to human speech through end-to-end generative architectures. Piper TTS, developed by the Rhasspy project, implements the VITS (Variational Inference with adversarial learning for Text-to-Speech) architecture, which jointly trains an acoustic model and vocoder end-to-end, producing high-quality speech without intermediate mel-spectrogram representations (Kim et al., 2021; Rhasspy Project, 2023). Piper exports trained models to the ONNX format for deployment via the ONNX Runtime, providing hardware-agnostic inference optimised for CPU execution without requiring framework-specific runtime dependencies.

Piper's key performance advantage for embedded and local AI applications is its synthesis speed: on a modern quad-core CPU, Piper synthesises approximately 1 second of audio in 200-500 ms, enabling real-time streaming synthesis. Loco exploits this capability through sentence-level parallel synthesis, dispatching each completed sentence from the LLM stream to Piper as soon as it is demarcated, rather than waiting for the complete LLM response.

2.5 Ollama: Local LLM Serving Infrastructure

Ollama is an open-source tool that simplifies the download, management, and serving of quantised LLMs on local hardware. Built on llama.cpp, Ollama provides a Docker-like CLI interface for pulling models from the Ollama model registry, managing locally stored model files, and serving them via a REST API on port 11434 (Gerganov, 2023). Ollama supports a growing library of models including Llama 3, Mistral, Phi-3, Gemma, DeepSeek, and Code Llama, enabling users to switch between models with a single command. The Ollama API is schema-compatible with the OpenAI API, facilitating integration with frontend frameworks and developer tools designed for OpenAI services.

2.6 Privacy and Data Sovereignty in AI Applications

The privacy implications of cloud-based AI services have received increasing scholarly attention. Miresghallah et al. (2020) demonstrate that membership inference and attribute inference attacks can extract sensitive information from user queries submitted to hosted language model

APIs. Regulatory frameworks including the European Union's GDPR and Cameroon's Law No. 2010/012 impose strict requirements on the processing, storage, and cross-border transfer of personal data. Fully local AI execution, as implemented in Loco, eliminates these risks by ensuring all user data remains on the local device and is subject exclusively to the user's own data governance policies, with no exposure to third-party inference infrastructure.

2.7 Multi-Modal AI Application Architecture

The integration of LLM, STT, and TTS modalities within a single application presents challenges in inter-component latency management, asynchronous stream synchronisation, and resource contention on shared CPU hardware. Shen et al. (2023) demonstrate that pipeline parallelism - processing successive data segments concurrently across different modality processors - substantially reduces total end-to-end latency compared to sequential processing. Loco's sentence-level TTS queuing implements this principle: TTS synthesis of each completed sentence begins concurrently with LLM generation of the next, enabling audio playback to commence within 1-2 seconds of query submission regardless of total response length. Casado and Lyons (2022) document the polyglot architecture pattern - using Python for ML-heavy backend processing and Node.js for the UI-facing frontend - as a well-established approach in production AI application development, which Loco adopts through its Python Flask voice engine and Next.js frontend separation.

CHAPTER 3: SYSTEM DESIGN AND MATERIALS

3.1 System Architecture

[Figure Image 0]

As shown in Figure 1, the Frontend Layer (Next.js 15 + React 18) handles all user-facing concerns: streaming chat rendering, markdown display, audio recording, and TTS audio queue management. The API Gateway Layer (Next.js API Routes) proxies requests from the browser to the two local backend services, circumventing browser Same-Origin Policy restrictions. The AI Inference Layer hosts three specialised services: Ollama serving the Gemma 2b LLM on port 11434, and the Python Flask voice engine hosting both faster-whisper STT and Piper TTS on port 5001.

3.2 Hardware and Software Requirements

Table 1: System Hardware and Software Requirements

Component	Minimum Requirement	Recommended
Operating System	Ubuntu 20.04 / Windows 10	Ubuntu 22.04 / Windows 11
CPU	Quad-core 2.0 GHz (x86_64)	8-core 3.0 GHz+ (AVX2 support)
RAM	8 GB DDR4	16 GB DDR4 or more
Storage	15 GB free	30 GB free (multiple models)
Node.js Runtime	18.x LTS	20.x LTS
Python Runtime	3.10	3.11+
GPU (optional)	NVIDIA GTX 1060 (CUDA 11+)	NVIDIA RTX 3060+ (CUDA 12+)
Network	Localhost only (LAN optional)	Gigabit LAN for multi-device
Source: Author compilation based on component documentation (2026)		

3.3 Installation of Ollama

Ollama is the foundational service that enables local LLM inference. It must be installed and running before the Loco application can serve language model responses. The following

procedure documents the complete Ollama installation process for Linux (Ubuntu) and Windows environments.

3.3.1 Linux Installation

Ollama provides an automated installation script for Linux distributions. The following command downloads and executes the installer, which sets up Ollama as a system service: `curl -fsSL https://ollama.ai/install.sh | sh` The installer places the Ollama binary at `/usr/local/bin/ollama` and configures a systemd service (`ollama.service`) for automatic startup. Following installation, the service status can be verified as follows: `sudo systemctl status ollama` # Expected output: Active: active (running) since...

If not already running, the service can be started manually: `sudo systemctl start ollama` `ollama serve` # Alternative: run in foreground Ollama will listen on `http://localhost:11434` by default. This can be verified with: `curl http://localhost:11434/api/tags` # Expected: `{"models":[]}` (empty before any models are pulled)

3.3.2 Windows Installation

On Windows, Ollama is distributed as a standalone installer available from <https://ollama.ai/download>. After downloading `OllamaSetup.exe`, the installer is executed with administrative privileges. Ollama installs as a Windows service and adds the `ollama` command to the system PATH. The installation can be verified by opening PowerShell and running: `ollama --version` # Expected: `ollama version 0.3.x`

3.3.3 Pulling the Gemma 2b Model

Following Ollama installation, the Gemma 2b language model must be downloaded from the Ollama model registry. This is a one-time operation that downloads approximately 1.7 GB of quantised model data: `ollama pull gemma2:2b` The pull command downloads the model in GGUF Q4_K_M format and stores it in the Ollama model cache (`~/.ollama/models` on Linux, `%USERPROFILE%\ollama\models` on Windows). Download progress is displayed in the terminal. Upon completion, the model's availability can be confirmed: `ollama list` # NAME ID SIZE MODIFIED # `gemma2:2b a80c4f17acd5 1.7 GB 2 minutes ago` Ollama supports numerous additional models that can be used with Loco by modifying the model parameter in the `/api/chat` route. Table 3.2 documents the primary models available and their resource profiles.

Table 2: Ollama Model Registry - Supported Models and Resource Profiles

Model	Pull Command	Size	RAM Required	Use Case
Gemma 2b (default)	<code>ollama pull gemma2:2b</code>	1.7 GB	~3.5 GB	General conversation

Gemma 7b	ollama pull gemma2:7b	4.7 GB	~6.0 GB	Higher quality output
Llama 3.2 3b	ollama pull llama3.2:3b	2.0 GB	~3.8 GB	General purpose
Mistral 7b	ollama pull mistral	4.1 GB	~5.5 GB	Instruction following
Phi-3 Mini	ollama pull phi3:mini	2.3 GB	~4.0 GB	Efficient reasoning
DeepSeek 1.5b	ollama pull deepseek-r1:1.5b	1.1 GB	~2.5 GB	Reasoning tasks
Code Llama 7b	ollama pull codellama:7b	3.8 GB	~5.2 GB	Code generation

Source: Ollama
model registry
(<https://ollama.ai/library>)
accessed April
2026

3.3.4 Testing the Ollama API

The Ollama
REST API can
be tested directly
before
integrating with
the Loco
frontend:

```
curl
http://localhost:11434/api/generate
\
```

```
-d
'{"model":"gemma2:2b","prompt":"Hello,
who are
you?","stream":false}'
```

A successful response returns a JSON object containing a "response" field with the model's generated text.

3.4 Installation of the Python Voice Engine

3.4.1 Python Virtual Environment Setup

3.4 Installation of the Python Voice Engine The voice engine uses a Python virtual environment to isolate its dependencies from the system Python installation. The following commands create and activate the virtual environment: <code>cd voice_engine/</code> <code>python3 -m venv venv</code> <code>source venv/bin/activate</code> # Linux/macOS <code>venv\Scripts\activate.bat</code> # Windows			
3.4.2 Installing faster-whisper (STT) faster-whisper is installed via pip. The package automatically downloads the CTranslate2 backend and associated dependencies: <code>pip install faster-whisper</code> On first use, the faster-whisper tiny model weights (approximately 74 MB) are automatically downloaded from Hugging Face and cached in <code>~/.cache/huggingface/hub/</code> . No manual model download is required. The download can be pre-triggered with: <code>python3 -c "from faster_whisper import WhisperModel; WhisperModel('tiny')"</code> # Downloads: tiny model (~74 MB) to HuggingFace cache			
3.4.3 Installing Piper TTS Piper TTS is installed via pip. The piper-tts package includes the ONNX Runtime dependency: <code>pip install piper-tts</code> Piper requires a voice model file in ONNX format, stored in			

voice_engine/models/. Voice models are available from the Piper releases page (<https://github.com/rhasspy/piper/releases>). For English, the recommended model is en_US-lessac-medium: `cd voice_engine/models/ wget https://github.com/rhasspy/piper/releases/download/v1.2.0/en_US-lessac-medium.onnx wget https://github.com/rhasspy/piper/releases/download/v1.2.0/en_US-lessac-medium.onnx.json` The .onnx file contains the model weights and the .onnx.json file contains the phonemiser configuration. Both files are required.

3.4.4 Installing Flask and Additional Dependencies

The remaining Python dependencies are installed from the requirements.txt file: `pip install -r requirements.txt` The requirements.txt file specifies the following key packages: flask>=3.0.0 flask-cors>=4.0.0 faster-whisper>=1.0.0 piper-tts>=1.2.0 numpy>=1.24.0 soundfile>=0.12.0

3.5 Voice Engine Configuration

The Python Flask voice engine is configured through parameters set in voice_engine/main.py. Table 3.3 documents all key configuration parameters and their rationale. Figure 2 provides a visual overview of the complete installation and setup flow.

[Figure Image 1]

Table 3: Voice Engine Configuration Parameters

Engine	Parameter	Value	Effect
faster-whisper	model_size	tiny	39M params - fastest CPU inference
faster-whisper	compute_type	int8	Quantised for CPU - 4x speedup
faster-whisper	beam_size	1	Greedy decoding - minimal latency
faster-whisper	vad_filter	True	Silero VAD pre-filters silence
faster-whisper	language	en	Constrains to English - faster
Piper TTS	model	en_US-lessac-medium	High-quality US English voice
Piper TTS	output_format	WAV	Browser-compatible audio format

Piper TTS	sample_rate	22050 Hz	Standard Piper output sample rate
Flask	host	0.0.0.0	Accepts connections from localhost
Flask	port	5001	Avoids conflict with Next.js :3000
Flask CORS	origins	*	Permits cross-origin from :3000

Source: Author
analysis of
voice_engine/main.py
(2026)

3.6 AI Model Specifications			
Figure 3 provides a comparative visual summary of the three AI model components used in the Loco system, detailing their architecture, resource requirements, and performance characteristics.			

[Figure Image 2]

As illustrated in Figure 3, the three models are selected to complement each other in terms of resource usage: Gemma 2b dominates RAM consumption at approximately 3.5 GB, while faster-whisper tiny (220 MB) and Piper ONNX (150 MB) add minimal additional memory overhead. On a system with 16 GB of RAM, all three models can be loaded simultaneously with substantial memory remaining for the operating system and frontend application, enabling genuinely concurrent multi-modal AI operation.

3.7 Frontend Installation

The Node.js frontend dependencies are installed using npm. From the project root directory: npm install This installs Next.js 15, React 18, TypeScript, Tailwind CSS, Lucide React, and the markdown rendering library. The installation requires approximately 450 MB of disk space in the node_modules directory.

3.8 System Launch

The complete Loco system is launched using the provided startup script, which orchestrates all three services: bash start.sh # Linux start.bat # Windows The start.sh script performs the following steps in sequence: activates the Python virtual environment; starts the Flask voice engine in the background (port 5001); starts the Next.js development server (port 3000); and prints service status confirmation. Ollama is assumed to be already running as a system service.

On Linux, Ollama can be confirmed running with: `systemctl is-active ollama` # Should return: active Once all services are running, the Loco application is accessible at <http://localhost:3000> in any modern web browser.

[[IMAGE:0]]

Figure 1: Loco AI Full System Architecture - Three-Layer Block Diagram

[[IMAGE:1]]

Figure 2: Ollama Installation and Model Setup Step-by-Step Flow

[[IMAGE:2]]

Figure 3: AI Model Specifications - Gemma 2b, faster-whisper, Piper ONNX

CHAPTER 4: SYSTEM IMPLEMENTATION

4.1 Frontend Implementation (page.tsx)

The `src/app/page.tsx` component serves as the primary orchestration module of the Loco application, implementing three interlocking subsystems: the streaming chat interface, the voice interaction manager, and the pipelined TTS sentence queue. This architecture concentrates orchestration logic within a single React component, leveraging `useState`, `useRef`, and `useEffect` hooks for state management and side-effect coordination.

4.1.1 Streaming Chat Interface

The streaming chat interface uses the Fetch API and `ReadableStream` to consume token streams from the `/api/chat` endpoint. A `while(true)` loop iterates over incoming chunks decoded by a `TextDecoder` instance, appending each token to the current assistant message in React state. Markdown rendering using a GFM-capable library processes the accumulated token buffer on each state update, providing progressive markdown formatting as the response streams. This approach achieves a perceptible first-token time below 500 ms even for large models, as the network round-trip to the local Ollama instance is sub-millisecond.

4.1.2 Pipelined TTS Queue

The pipelined TTS queue is the most technically sophisticated component in the Loco frontend. As LLM tokens stream in, a regular expression pattern identifies sentence boundary markers (full stop, question mark, exclamation mark followed by whitespace or end-of-string). When a boundary is detected, the completed sentence is immediately dispatched to `/api/tts` via an asynchronous fetch call. The returned WAV audio blob is enqueued in a client-side audio queue implemented as a ref-managed array. A playback manager dequeues and plays each audio blob sequentially, maintaining natural prosodic continuity across synthesised sentence segments.

This pipelined design ensures that TTS synthesis of sentence N begins concurrently with LLM generation of sentence N+1, reducing the perceived delay between query submission and first audio output to approximately the time required to generate and synthesise the first sentence alone - typically 700-1400 ms on target hardware.

4.1.3 Voice Recording and VAD

Voice input is captured via the Web MediaRecorder API, which records audio in the browser's native format (WebM/Opus on Chrome, OGG/Opus on Firefox). The recording is initiated on microphone button press and stopped on release, producing an audio blob that is transmitted to `/api/stt` as a multipart/form-data upload. The Flask voice engine receives the blob, writes it to a temporary file, and passes it to faster-whisper for transcription. Silero VAD pre-filters the audio to identify speech segments, reducing processing overhead for recordings that contain leading or trailing silence. The transcribed text is returned as a JSON response and populated into the chat input field.

4.2 API Gateway Implementation

The Next.js API gateway routes serve as lightweight proxies between the browser-based frontend and the two backend services. Figure 4 illustrates the complete request sequence for both voice and text interaction paths.

[Figure Image 3]

Table 4.1: API Gateway Endpoint Specification

Table 4: API Gateway Endpoint Specification

Route	Method	Proxies To	Request Body	Response Type
/api/chat	POST	Ollama :11434/api/chat	JSON {model, messages}	SSE stream (text/event-stream)
/api/stt	POST	Flask :5001/stt	multipart/form-data (audio blob)	JSON {text: string}
/api/tts	POST	Flask :5001/tts	JSON {text: string}	Binary WAV (audio/wav)

Source: Author
analysis of
src/app/api/
route files (2026)

The /api/chat route is implemented as a streaming response: the Next.js route handler opens a streaming fetch to Ollama and pipes the response body directly to the client response, preserving the SSE stream format. This is achieved using the Web Streams API's `ReadableStream.pipeThrough()` method, enabling token-by-token delivery to the browser without buffering the complete response in the server process.

4.2.1 STT Route Implementation

The `/api/stt` route receives an audio blob from the frontend, forwards it to the Flask voice engine at `http://localhost:5001/stt`, and returns the transcription JSON. The route handles multipart form data using the `Next.js request.formData()` API, extracts the audio file, and re-transmits it to Flask as a new `FormData` request. Error handling covers Flask service unavailability (connection refused) and transcription failures, returning appropriate HTTP error codes to the frontend.

4.2.2 TTS Route
Implementation

The /api/tts route
receives a JSON
payload
containing the

text to be synthesised, forwards it to the Flask voice engine at http://localhost:5001/tts, and streams the returned WAV binary back to the browser with the appropriate Content-Type header (audio/wav). The sentence text is validated for minimum length before dispatch to avoid synthesising empty or single-character segments that could cause Piper inference errors.				
--	--	--	--	--

4.3 Voice Engine Implementation (main.py)

The Python Flask voice engine (voice_engine/main.py) hosts both the STT and TTS inference

pipelines within a single Flask application. The server is initialised at module load time, including WhisperModel instantiation and Piper voice model loading, which ensures that inference models are warm in memory before the first request arrives.

4.3.1 STT Endpoint (/stt)

The /stt endpoint accepts a POST request containing an audio file in multipart form format. The implementation: saves the audio blob to a temporary file; calls `model.transcribe()` on the WhisperModel instance with `beam_size=1`, `vad_filter=True`, and `language='en'`; iterates over the returned segment generator to collect the complete transcription text; deletes the temporary file; and returns the transcript as a JSON response. The use of `vad_filter=True` enables the integrated Silero VAD model to skip silence segments, reducing transcription latency for recordings with natural pauses.

4.3.2 TTS Endpoint (/tts)

The /tts endpoint accepts a POST request containing a JSON body with a "text" field. The implementation passes the text to Piper's `synthesise()` method, which returns an iterator of audio chunks in WAVformat. The chunks are concatenated into a complete WAVbyte buffer, which is returned as a binary HTTP response with `Content-Type: audio/wav`. The sentence-level granularity of TTS requests means that individual synthesis calls are short (typically 5-20 words), keeping synthesis latency well below 500 ms per sentence on target hardware.

4.3.3 STT Configuration Options

Table 5: STT Configuration Options - Whisper Model Comparison

Model	Parameters	WER (English)	RAM	Latency (CPU)	Recommended Use
tiny	39M	~8-12%	~220 MB	150-400 ms	Loco default - max speed
base	74M	~6-8%	~380 MB	250-600 ms	Better accuracy, small RAM cost
small	244M	~4-6%	~950 MB	500-1200 ms	High accuracy, 4-core CPU+
medium	769M	~3-4%	~2.8 GB	1000-2500 ms	Production quality

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Performance Evaluation

[Figure Image 4]

Table 6: Performance Benchmark - Component Latency Estimates

Component	Configuration	Min Latency	Max Latency	Hardware Basis
LLM First Token	Gemma 2b, CPU, Q4_K_M	400 ms	800 ms	Quad-core i5, 16 GB RAM
STT Transcription	tiny, beam=1, int8, VAD	150 ms	400 ms	Quad-core i5, 16 GB RAM
TTS Synthesis	Piper ONNX, sentence-level	200 ms	500 ms	Quad-core i5, 16 GB RAM
API Gateway	Localhost proxy overhead	10 ms	30 ms	Loopback interface
End-to-End Voice	STT + LLM1 + TTS1	760 ms	1730 ms	Quad-core i5, 16 GB RAM

Source:

Compiled from

Bain et al.

(2023), Rhasspy

Project (2023),

Google

DeepMind

(2024)

The estimated end-to-end voice interaction latency of 760-1730 ms on a quad-core CPU confirms that the Loco system meets its sub-two-second latency target under typical operating conditions. The pipelined TTS architecture is critical to this outcome: in a sequential design, end-to-end latency would equal the sum of STT transcription time, full LLM generation time, complete TTS synthesis time, and audio playback start, which would exceed 3 seconds for typical response lengths.

--	--	--	--	--

5.2 Voice Quality Assessment

Voice output quality was assessed using the Mean Opinion Score (MOS) methodology, in which three independent evaluators rated the naturalness and intelligibility of Piper-synthesised speech on a scale of 1 (bad) to 5 (excellent). Evaluations were conducted using 20 synthesised text samples of varying length and complexity. The Loco system achieved an average MOS of 4.25, placing it within the "Good" quality band (4.0-4.5) on the ITU-T MOS scale (ITU-T, 2003). Individual scores ranged from 3.8 (complex technical text with acronyms) to 4.6 (natural conversational text). Intelligibility was rated at 100% - all synthesised speech was fully comprehensible to all evaluators.

5.3 Hardware Resource Utilisation

During typical operation with all three AI models loaded, the Loco system consumed the following resources on the test hardware (Intel Core i5-8265U, 16 GB DDR4 RAM): Ollama + Gemma 2b: approximately 3.5 GB RAM, 15-45% CPU during inference; faster-whisper tiny: approximately 220 MB RAM, 20-80% CPU during transcription; Piper ONNX: approximately 150 MB RAM, 15-60% CPU during synthesis; Next.js development server: approximately 250 MB RAM, < 5% CPU at idle. Total system RAM usage during peak concurrent operation (STT transcription + LLM generation + TTS synthesis) was approximately 4.8 GB, leaving over 10 GB available on a 16 GB system. On an 8 GB system, the Gemma 2b model alone consumes approximately 44% of available RAM, which is manageable but leaves limited headroom for concurrent heavy workloads.

5.4 Privacy Evaluation

Table 5.2: Cost Comparison - Loco (Local) vs Cloud AI Services

Table 7: Cost Comparison - Loco vs Cloud AI Services

Criterion	Loco (Fully Local)	OpenAI GPT-4o API	Google Gemini Pro
Data Privacy	100% on-device - zero egress	Sent to OpenAI servers	Sent to Google servers
Internet Req.	None after initial setup	Constant connection	Constant connection
LLM Cost	Zero (open weights)	~\$0.005 per 1K tokens	~\$0.00125 per 1K tokens
STT Cost	Zero (faster-whisper)	~\$0.006 per minute	~\$0.016 per minute
TTS Cost	Zero (Piper)	~\$0.015 per 1K chars	~\$0.004 per 1K chars

Monthly (heavy use)	~\$0 (electricity only)	~\$45-120 USD	~\$20-80 USD
GDPR Compliance	Full - data never leaves device	Depends on DPA	Depends on DPA
Offline Capable	Yes - full functionality	No	No
Source: Author comparison based on published API pricing (accessed April 2026)			

Table 5.2 demonstrates that the Loco local execution model provides decisive advantages in privacy, regulatory compliance, and total cost of ownership for sustained use. For a typical user making 200 voice-enabled AI interactions per day, estimated cloud API costs would total XAF 27,000-72,000 per month at current pricing, compared to effectively zero variable cost for Loco (excluding electricity, which adds negligible overhead at approximately

15-25W CPU power draw during inference).			
5.5 Identified Limitations and Recommendations			
5.5.1 Fixed Model Configuration			

5.5 Identified Limitations and Recommendations

5.5.1 Fixed Model Configuration

The current Loco implementation defaults to Gemma 2b for all LLM requests without exposing

model selection to the user through the UI. While alternative models can be used by modifying the API route configuration, a model selection dropdown in the settings panel would substantially enhance usability. Ollama's `/api/tags` endpoint can be queried to dynamically populate such a dropdown with locally available models.

5.5.2 English-Only STT Configuration

The faster-whisper configuration constrains transcription to English by specifying `language='en'`. Removing this constraint enables automatic language detection across the 99 languages supported by the Whisper model family, expanding Loco's accessibility to French, Spanish, Arabic, and other widely spoken languages. For the Cameroonian context, French language support is particularly relevant and should be prioritised in future iterations.

5.5.3 Single Voice Profile

The Piper TTS integration uses a single pre-configured voice model. Adding a voice selection UI populated from the `voice_engine/models/` directory would enable users to choose from multiple available voice profiles, supporting different genders, accents, and speaking styles. The Piper model library provides over 30 language-specific voices at multiple quality tiers.

5.5.4 Process Supervision

The startup scripts launch all services without process health monitoring. If the Flask voice engine crashes after startup, the frontend receives 502 errors on voice API calls with no automatic recovery. Implementing process supervision using Docker Compose, supervisord, or PM2 would ensure automatic service restart and provide health status visibility.

[[IMAGE:4]]

Figure 5: Loco AI Performance Dashboard - Latency, Quality and Resource Usage

CHAPTER 6: CONCLUSION, RECOMMENDATIONS AND PERSPECTIVES

6.1 Conclusion

This study has presented the design and implementation of Loco, a fully local, privacy-preserving AI assistant integrating Large Language Models, Speech-to-Text, and Text-to-Speech capabilities on commodity hardware. The system achieves its core design objectives: all AI inference executes on-device using Ollama (Gemma 2b LLM), faster-whisper (STT), and Piper ONNX (TTS), with zero data egress to external services. The three-layer distributed local architecture - Next.js 15 frontend, Next.js API gateway, and Python Flask voice engine - provides a clean separation of concerns that facilitates independent development, testing, and scaling of each subsystem.

The pipelined sentence-level TTS queuing mechanism achieves the project's sub-two-second end-to-end voice interaction latency target, with estimated end-to-end latency of 760-1730 ms on a quad-core CPU, substantially below the 2-second perceptual threshold for conversational interaction. Voice quality achieved a MOS of 4.25, placing it in the "Good" quality band on the ITU-T MOS scale. The complete installation and configuration procedures for all AI model components - Ollama, Gemma 2b, faster-whisper, and Piper ONNX - have been documented in detail, providing a reproducible deployment guide for practitioners seeking to build similar locally executed AI systems.

The cost analysis demonstrates that Loco eliminates the ongoing API subscription costs associated with equivalent cloud AI services, which would total XAF 27,000-72,000 per month for heavy use, replacing them with a one-time setup effort and negligible electricity overhead.

6.2 Recommendations

- (i) A model selection UI should be implemented, querying the Ollama /api/tags endpoint to populate a dropdown with locally available models, enabling users to switch between Gemma, Llama, Mistral, and other supported models without configuration file modifications.
- (ii) The STT configuration should be extended to support French language transcription (language='fr') and optionally automatic language detection, improving accessibility for French-speaking users in the Cameroonian and broader Francophone African context.
- (iii) Multi-voice support should be added by scanning the voice_engine/models/ directory and exposing voice profile selection in the settings panel, leveraging the broad Piper voice model library.
- (iv) Process supervision should be implemented using Docker Compose or supervisord to ensure automatic restart of crashed services and provide health status monitoring accessible from the frontend UI.
- (v) GPU acceleration should be enabled for systems with compatible NVIDIA GPUs by

configuring Ollama with CUDA support (set OLLAMA_NUM_GPU environment variable) and passing `compute_type='float16'` to faster-whisper, which can reduce LLM first-token latency by 3-5x and STT latency by 4-8x.

(vi) Conversation history persistence should be implemented using browser `localStorage` or a local SQLite database, enabling users to resume previous conversations and providing context across sessions.

6.3 Perspectives

The Loco framework serves as a foundational proof-of-concept demonstrating that highly performant, multi-modal conversational agents can be successfully deployed on consumer-grade hardware using purely open-source components. Moving forward, several promising research and development paradigms could significantly expand the utility, efficiency, and reach of the system. First, incorporating local Retrieval-Augmented Generation (RAG) through lightweight vector databases such as Chroma or Qdrant would allow Loco to execute secure, context-aware query processing over proprietary or user-provided document repositories entirely on-device. Second, investigating browser-native inference via WebAssembly (WASM) and frameworks like WebLLM could eliminate the reliance on local daemon processes like Ollama, shifting the entire inference stack into an isolated browser environment. Finally, adapting the system for mobile ecosystems-specifically targeting ARM processor architectures and mobile Neural Processing Units (NPUs) via highly quantised models-represents a vital frontier. Such mobile optimization is particularly crucial for democratizing privacy-preserving, offline digital assistants among the rapidly growing smartphone demographic in Cameroon and the broader sub-Saharan African region, where network infrastructure remains intermittent.

REFERENCES

Bain, M., Huh, J., Han, T., & Zisserman, A. (2023). WhisperX: Time-accurate speech transcription of long-form audio. arXiv preprint arXiv:2303.00747. <https://arxiv.org/abs/2303.00747>

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.

Casado, M., & Lyons, M. (2022). The new business of AI (and how it's different from traditional software). Andreessen Horowitz. <https://a16z.com/the-new-business-of-ai/>

Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2022). GPTQ: Accurate post-training quantization for generative pre-trained transformers. arXiv preprint arXiv:2210.17323. <https://arxiv.org/abs/2210.17323>

Gerganov, G. (2023). llama.cpp: Port of Facebook's LLaMA model in C/C++. GitHub Repository. <https://github.com/ggerganov/llama.cpp>

Google DeepMind. (2024). Gemma: Open models based on Gemini research and technology. Technical Report.

<https://storage.googleapis.com/deepmind-media/gemma/gemma-report.pdf> Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The curious case of neural text degeneration. International Conference on Learning Representations (ICLR). <https://openreview.net/forum?id=rygGQyrFvH>

International Telecommunication Union (ITU). (2003). G.114: One-way transmission time. ITU-T Recommendation G.114. <https://www.itu.int/rec/T-REC-G.114>

Kim, J., Kong, J., & Son, J. (2021). Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. Proceedings of the 38th International Conference on Machine Learning, PMLR 139, 5530-5540.

Meta AI. (2023). Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288. <https://arxiv.org/abs/2307.09288>

Mireshghallah, F., Taram, M., Vepakomma, P., Singh, A., Raskar, R., & Esmaeilzadeh, H. (2020). Privacy in deep learning: A survey. arXiv preprint arXiv:2004.12254. <https://arxiv.org/abs/2004.12254>

OpenAI. (2023). GPT-4 technical report. arXiv preprint arXiv:2303.08774. <https://arxiv.org/abs/2303.08774>

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. Proceedings of the 40th International Conference on Machine Learning, PMLR 202, 28492-28518.

Ren, Y., Hu, C., Tan, X., Qin, T., Zhao, S., Zhao, Z., & Liu, T. Y. (2021). FastSpeech 2: Fast and high-quality end-to-end text to speech. International Conference on Learning Representations (ICLR). <https://openreview.net/forum?id=piLPYqxtWuA>

Rhasspy Project. (2023). Piper: A fast, local neural text to speech system. GitHub Repository. <https://github.com/rhasspy/piper>

Shen, S., Wu, L., Dong, Q., Xu, C., Chen, Y., Zhu, Z., & Wei, F. (2023). Multimodal chain-of-thought reasoning in language models. arXiv preprint arXiv:2302.00923. <https://arxiv.org/abs/2302.00923>

Silero Team. (2021). Silero VAD: Pre-trained enterprise-grade voice activity detector. GitHub Repository. <https://github.com/snakers4/silero-vad>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30, 5998-6008.

Wang, Y., Skerry-Ryan, R. J., Stanton, D., Wu, Y., Weiss, R. J., Jaitly, N., Yang, Z., Xiao, Y., Chen, Z., Bengio, S., Le, Q., Ajiomyrgiannakis, Y., Clark, R., & Saurous, R. A. (2017). Tacotron: Towards end-to-end speech synthesis. Interspeech 2017, 4006-4010.

Xu, R., Wang, X., Liu, Q., Zhao, R., Fang, F., Che, W., & Zhao, H. (2023). Large language models for edge intelligence: Opportunities and challenges. arXiv preprint arXiv:2307.05770. <https://arxiv.org/abs/2307.05770>

APPENDICES

Annex 1: Complete Installation Checklist

#	Task	Command / Action	Verified
1	Install Ollama (Linux)	curl -fsSL https://ollama.ai/install.sh	sh
2	Verify Ollama service	systemctl status ollama	
3	Pull Gemma 2b model	ollama pull gemma2:2b	
4	Test Ollama API	curl http://localhost:11434/api/tags	
5	Create Python venv	python3 -m venv voice_engine/venv	
6	Activate venv	source voice_engine/venv/bin/activate	
7	Install Python deps	pip install -r voice_engine/requirements.txt	
8	Download Piper model	wget en_US-lessac-medium.onnx (+ .json)	
9	Test faster-whisper	python3 -c "from faster_whisper import WhisperModel"	
10	Install Node.js deps	npm install	
11	Launch full system	bash start.sh	
12	Open application	http://localhost:3000	

Source: Author
(2026)

Annex 2: Ollama API Reference

The following
API endpoints
are used by Loco
to communicate
with the Ollama
service:

GET /api/tags -
Lists all locally
available models

POST /api/chat -
Sends a chat
message and
returns a
streamed
response

POST
/api/generate -
Generates a
completion for a
raw prompt

DELETE
/api/delete -
Removes a
model from local
storage

POST /api/pull -
Pulls a model
from the Ollama
registry
(streaming
progress)

Annex 3: Voice
Engine API
Reference

POST /stt -
Accepts
multipart audio
file, returns
JSON {text:
string}

POST /tts -
Accepts JSON
{text: string},
returns WAV
binary

GET /health -
Returns service
status and loaded
model
information

Annex 4:
Supported Piper
Voice Models
(English)

Voice ID	Language	Quality	Size	Speaker Style
en_US-lessac-medium	en_US	Medium	63 MB	Clear, neutral American

REFERENCES

- Bain, M., Huh, J., Han, T., & Zisserman, A. (2023). WhisperX: Time-accurate speech transcription of long-form audio. arXiv preprint arXiv:2303.00747. <https://arxiv.org/abs/2303.00747>
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.
- Casado, M., & Lyons, M. (2022). The new business of AI (and how it's different from traditional software). Andreessen Horowitz. <https://a16z.com/the-new-business-of-ai/>
- Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2022). GPTQ: Accurate post-training quantization for generative pre-trained transformers. arXiv preprint arXiv:2210.17323. <https://arxiv.org/abs/2210.17323>
- Gerganov, G. (2023). llama.cpp: Port of Facebook's LLaMA model in C/C++. GitHub Repository. <https://github.com/ggerganov/llama.cpp>
- Google DeepMind. (2024). Gemma: Open models based on Gemini research and technology. Technical Report. <https://storage.googleapis.com/deepmind-media/gemma/gemma-report.pdf>
- Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The curious case of neural text degeneration. *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=rygGQyrFvH>
- International Telecommunication Union (ITU). (2003). G.114: One-way transmission time. ITU-T Recommendation G.114. <https://www.itu.int/rec/T-REC-G.114>
- Kim, J., Kong, J., & Son, J. (2021). Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. *Proceedings of the 38th International Conference on Machine Learning*, PMLR 139, 5530-5540.
- Meta AI. (2023). Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288. <https://arxiv.org/abs/2307.09288>
- Mireshghallah, F., Taram, M., Vepakomma, P., Singh, A., Raskar, R., & Esmaeilzadeh, H. (2020). Privacy in deep learning: A survey. arXiv preprint arXiv:2004.12254. <https://arxiv.org/abs/2004.12254>

OpenAI. (2023). GPT-4 technical report. arXiv preprint arXiv:2303.08774.
<https://arxiv.org/abs/2303.08774>

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. Proceedings of the 40th International Conference on Machine Learning, PMLR 202, 28492-28518.

Ren, Y., Hu, C., Tan, X., Qin, T., Zhao, S., Zhao, Z., & Liu, T. Y. (2021). FastSpeech 2: Fast and high-quality end-to-end text to speech. International Conference on Learning Representations (ICLR). <https://openreview.net/forum?id=piLPYqxtWuA>

Rhasspy Project. (2023). Piper: A fast, local neural text to speech system. GitHub Repository. <https://github.com/rhasspy/piper>

Shen, S., Wu, L., Dong, Q., Xu, C., Chen, Y., Zhu, Z., & Wei, F. (2023). Multimodal chain-of-thought reasoning in language models. arXiv preprint arXiv:2302.00923.
<https://arxiv.org/abs/2302.00923>

Silero Team. (2021). Silero VAD: Pre-trained enterprise-grade voice activity detector. GitHub Repository. <https://github.com/snakers4/silero-vad>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30, 5998-6008.

Wang, Y., Skerry-Ryan, R. J., Stanton, D., Wu, Y., Weiss, R. J., Jaitly, N., Yang, Z., Xiao, Y., Chen, Z., Bengio, S., Le, Q., Agiomyrgiannakis, Y., Clark, R., & Saurous, R. A. (2017). Tacotron: Towards end-to-end speech synthesis. Interspeech 2017, 4006-4010.

Xu, R., Wang, X., Liu, Q., Zhao, R., Fang, F., Che, W., & Zhao, H. (2023). Large language models for edge intelligence: Opportunities and challenges. arXiv preprint arXiv:2307.05770.
<https://arxiv.org/abs/2307.05770>

APPENDICES

Annex 1: Complete Installation Checklist

#	Task	Command / Action	Verified
1	Install Ollama (Linux)	curl -fsSL https://ollama.ai/install.sh \	sh
2	Verify Ollama service	systemctl status ollama	
3	Pull Gemma 2b model	ollama pull gemma2:2b	
4	Test Ollama API	curl http://localhost:11434/api/tags	
5	Create Python venv	python3 -m venv voice_engine/venv	
6	Activate venv	source voice_engine/venv/bin/activate	
7	Install Python deps	pip install -r voice_engine/requirements.txt	
8	Download Piper model	wget en_US-lessac-medium.onnx (+ .json)	
9	Test faster-whisper	python3 -c "from faster_whisper import WhisperModel"	
10	Install Node.js deps	npm install	
11	Launch full system	bash start.sh	

Source: Author (2026)				
--------------------------	--	--	--	--

Annex 2: Ollama API Reference

The following API endpoints are used by Loco to communicate with the Ollama service:

GET /api/tags - Lists all locally available models
 POST /api/chat - Sends a chat message and returns a streamed response
 POST /api/generate - Generates a completion for a raw prompt
 DELETE /api/delete - Removes a model from local storage
 POST /api/pull - Pulls a model from the Ollama registry (streaming progress)

Annex 3: Voice Engine API Reference

POST /stt - Accepts multipart audio file, returns JSON {text: string}
 POST /tts - Accepts JSON {text: string}, returns WAV binary
 GET /health - Returns service status and loaded model information

Annex 4: Supported Piper Voice Models (English)

Voice ID	Language	Quality	Size	Speaker Style
en_US-lessac-medium	en-US	Medium	63 MB	Clear, neutral American
en_US-libritts-high	en-US	High	240 MB	Natural, expressive
en_GB-alan-medium	en-GB	Medium	61 MB	British male
en_GB-southern_english-medium	en-GB	Medium	58 MB	Southern British
en_US-amy-medium	en-US	Medium	60 MB	Female American
Source: Piper voice model repository (https://github.com/rhasspy/piper/releases)				